



資料結構

Data Structure

Lab 11

姓名： 陳冠豪

學號： 114310125

Lab11-Q1

Complement Level Sum function

Code

```
#include <iostream>
#include <queue>
#include <vector>
using namespace std;

const int EMPTY = NULL; //用 NULL 表示該位置沒有節點

//樹的節點類別
class TreeNode {
public:
    int value;          //儲存節點的值
    TreeNode* left;     //指向左子節點
    TreeNode* right;    //指向右子節點

    //建構子，建立節點時順便設定數值
    TreeNode(int val) : value(val), left(nullptr), right(nullptr) {}
};

//二元樹類別
class BinaryTree {
public:
    TreeNode* root; //根節點

    //一開始先建立空樹
    BinaryTree() : root(nullptr) {}

    //根據陣列內容建立二元樹
    TreeNode* buildTree(const vector<int>& arr) {

        //如果陣列是空的或第一個位置沒有資料
        //代表無法建立樹
        if (arr.empty() || arr[0] == EMPTY) return nullptr;
```

```

queue<TreeNode**> q;

// 建立根節點
root = new TreeNode(arr[0]);

// 將根節點放進 queue
q.push(&root);

size_t i = 1;

// 使用 BFS 的方式建立整棵樹
while (!q.empty() && i < arr.size()) {

    TreeNode** nodePtr = q.front();
    q.pop();

    // 建立左子節點
    if (i < arr.size()) {
        if (arr[i] != EMPTY) {
            (*nodePtr)->left = new TreeNode(arr[i]);

            // 把新節點加入 queue
            q.push(&((*nodePtr)->left));
        }
        i++;
    }

    // 建立右子節點
    if (i < arr.size()) {
        if (arr[i] != EMPTY) {
            (*nodePtr)->right = new TreeNode(arr[i]);

            // 把新節點加入 queue
            q.push(&((*nodePtr)->right));
        }
        i++;
    }
}

```

```

    }

    return root;
}

//DFS
//走訪順序：根 -> 左 -> 右
void Depth_first_search(TreeNode* node) {

    //遇到空節點就結束
    if (node == nullptr) return;

    cout << node->value << " ";

    //遞迴走訪左子樹
    Depth_first_search(node->left);

    //遞迴走訪右子樹
    Depth_first_search(node->right);
}

//BFS)
//依照層數逐層拜訪
void Breadth_first_search(TreeNode* root) {

    if (root == nullptr) return;

    queue<TreeNode*> q;

    //先把根節點放入 queue
    q.push(root);

    while (!q.empty()) {

        TreeNode* current = q.front();
        q.pop();

        cout << current->value << " ";
    }
}

```

```

        //左子節點加入 queue
        if (current->left)
            q.push(current->left);

        //右子節點加入 queue
        if (current->right)
            q.push(current->right);
    }
}

//計算指定層的節點總和
void levelSum(TreeNode* root, int layer) {

    if (root == nullptr) return;

    queue<TreeNode*> q;

    //從根節點開始
    q.push(root);

    int level = 0; //紀錄目前所在層數

    while (!q.empty()) {

        //取得目前層的節點數量
        int levelSize = q.size();

        //紀錄該層的總和
        int levelSumVal = 0;

        //依序處理同一層的所有節點
        for (int i = 0; i < levelSize; ++i) {

            TreeNode* current = q.front();
            q.pop();

            //如果是使用者指定的層數

```

```

        //就把節點值加總
        if (level == layer) {
            levelSumVal += current->value;
        }

        //將下一層的節點加入 queue
        if (current->left)
            q.push(current->left);

        if (current->right)
            q.push(current->right);
    }

    //找到目標層後直接輸出結果
    if (level == layer) {
        cout << "The sum of level "
              << layer
              << " is: "
              << levelSumVal
              << endl;

        return;
    }

    //準備進入下一層
    level++;
}

//輸入的層數超過樹的高度
cout << "The layer exceeds the tree height." << endl;
}
};

int main() {

    BinaryTree tree;

    //用陣列建立測試用二元樹

```

```

vector<int> arr = {
    1, 2, 3,
    4, 5, 6, 7,
    8, 9, NULL, NULL,
    10, 11, NULL, NULL
};

tree.buildTree(arr);

//顯示BFS 走訪結果
cout << "BFS Result: ";
tree.Breadth_first_search(tree.root);
cout << endl;

int layer;

//輸入想查詢的層數
cout << "Please enter the layer to query, starting from 0: ";
cin >> layer;

//計算該層節點總和
tree.levelSum(tree.root, layer);

system("pause");
return 0;
}

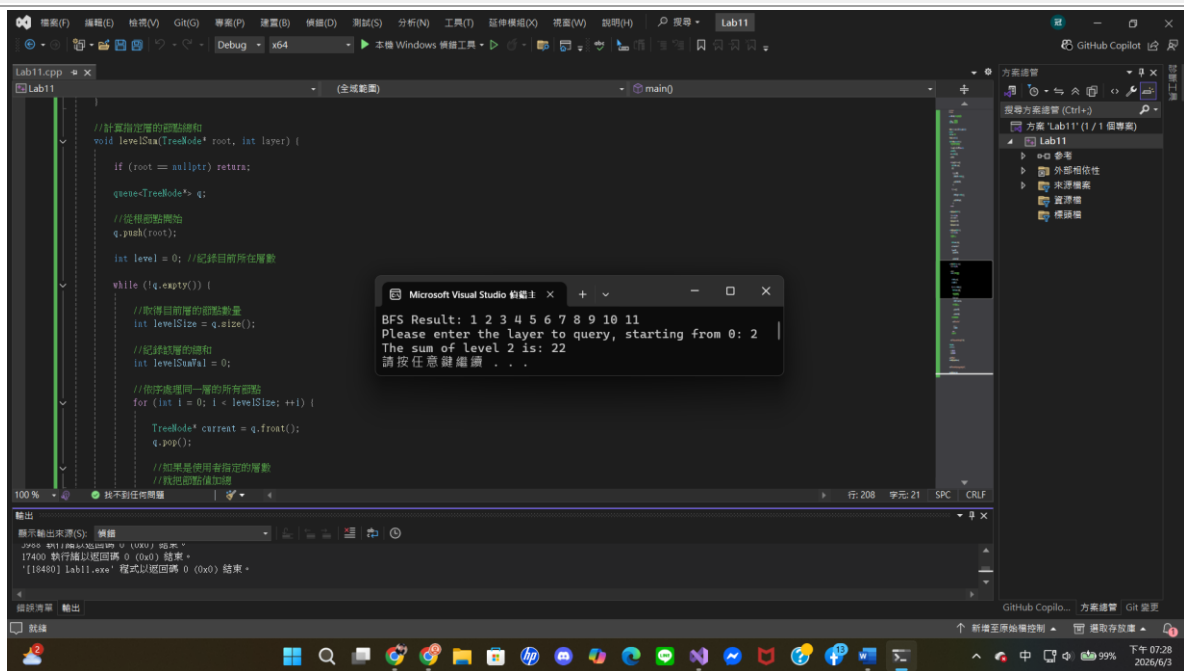
```

Result

```

BFS Result: 1 2 3 4 5 6 7 8 9 10 11
Please enter the layer to query, starting from 0: 2
The sum of level 2 is: 22
請按任意鍵繼續 . . .

```



BFS Result: 1 2 3 4 5 6 7 8 9 10 11
Please enter the layer to query, starting from 0: 4
The layer exceeds the tree height.
請按任意鍵繼續 . . .

